# Class Note: Functions in C

Nov 2, 2026 · @Hasan Sonet

**Course:** CSE 1112 : SPL (Structured Programming Language Laboratory)

A function is a named, reusable block of code that takes inputs, does one job and optionally returns a value; C programs are built from them, starting with `main()`.

## The big idea: a function is a vending machine

You put in coins and press a button (the **arguments**). The machine does its work out of sight (the **body**). A snack drops out (the **return value**). You never need to know how the machine works inside, only what it needs and what it gives back.

```
int snack = vend(50, 3);   // coins in, button pressed, snack out
```

That is how every function in C works, from `printf()` to the ones you write. Keep this picture in mind through the rest of the note.

## 1. Why use functions

Functions break a large program into small pieces that can be written, tested and reused separately.

- **Reusability:** write once, call many times.
- **Modularity:** each function does one job, so programs are easier to read and debug.
- **Less repetition:** a fix in one place fixes every call.
- **Teamwork:** different people can write different functions.

C has two kinds of functions:

| Kind | Defined by | Examples |
|---|---|---|
| Library (built-in) | C standard library, via a header | `printf()`, `scanf()` (`<stdio.h>`), `sqrt()` (`<math.h>`), `strlen()` (`<string.h>`) |
| User-defined | The programmer | `add()`, `factorial()`, `isPrime()` |

## 2. Anatomy of a function

Every function involves three parts: a declaration, a definition and a call.

```
#include <stdio.h>

int sum(int a, int b);          // 1. declaration (prototype)

int main(void) {
    int s = sum(5, 3);          // 3. call
    printf("Sum = %d\n", s);
    return 0;
}
```

```
int sum(int a, int b) {         // 2. definition
    return a + b;
}
```
Output: `Sum = 8`

| Part | Syntax | Purpose |
|---|---|---|
| Declaration (prototype) | `return_type name(parameter_list);` | Tells the compiler the function's name, parameter types and return type before it is used. Ends with `;`. |
| Definition | `return_type name(parameters) { body }` | Contains the actual code. |
| Call | `name(arguments);` | Transfers control to the function; the result replaces the call expression. |

- **Parameters** (formal) are the variables in the definition: `a`, `b`.
- **Arguments** (actual) are the values passed in the call: `5`, `3`.
- `return` sends a value back and ends the function immediately.
- A `void` return type means nothing is returned.

## 3. Four types of user-defined functions

Functions are classified by whether they take arguments and whether they return a value.

| Type | Arguments | Return value | Example prototype |
|---|---|---|---|
| 1 | No | No | `void greet(void);` |
| 2 | Yes | No | `void printSquare(int n);` |
| 3 | No | Yes | `int readNumber(void);` |
| 4 | Yes | Yes | `int max(int a, int b);` |

```
int max(int a, int b) {   // type 4: arguments and return value
    return (a > b) ? a : b;
}
```
Type 4 is the most useful, because the function does not depend on global data and its result can be reused in expressions.

## 4. Passing arguments

C passes arguments **by value**: the function receives a copy, so changing a parameter never changes the caller's variable. Functions cannot accidentally damage your data.

```
void doubleIt(int x) {
    x = x * 2;
    printf("inside: %d\n", x);
}
```

```
int main(void) {
    int n = 5;
    doubleIt(n);                  // prints inside: 10
    printf("outside: %d\n", n);   // prints outside: 5
    return 0;
}
```

## 5. Scope and storage classes

Scope is the region of code where a name is visible; storage class decides how long a variable lives.

| Kind | Declared | Visible in | Lifetime | Default value |
|---|---|---|---|---|
| Local (`auto`) | Inside a function | That function (or block) | Until the function returns | Garbage |
| Global | Outside all functions | Whole file after the declaration | Whole program | 0 |
| `static` local | Inside a function | That function | Whole program, value kept between calls | 0 |

```
void counter(void) {
    static int count = 0;   // initialised once
    count++;
    printf("%d ", count);
}
// calling counter() three times prints: 1 2 3
```

Prefer local variables and parameters; global variables make functions harder to test and reason about.

## 6. Using functions with arrays

An array is not copied when passed to a function: the function works on the **original** array. So a function can read it, and also change it. Always pass the size as a second argument, because the function cannot work out the length itself.

```
#include <stdio.h>

int sumArray(int arr[], int n);        // reads the array
int maxArray(int arr[], int n);        // reads the array
void doubleAll(int arr[], int n);      // changes the array
void printArray(int arr[], int n);

int main(void) {
    int marks[5] = {10, 20, 30, 40, 50};

    printf("Sum = %d\n", sumArray(marks, 5));   // 150
    printf("Max = %d\n", maxArray(marks, 5));   // 50

    doubleAll(marks, 5);
    printArray(marks, 5);                        // 20 40 60 80 100
    return 0;
}
```

```
int sumArray(int arr[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) total += arr[i];
    return total;
}

int maxArray(int arr[], int n) {
    int m = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > m) m = arr[i];
    return m;
}

void doubleAll(int arr[], int n) {
    for (int i = 0; i < n; i++) arr[i] = arr[i] * 2;
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}
```

| Task | Call | Result |
|---|---|---|
| Add all elements | `sumArray(marks, 5)` | 150 |
| Find the largest | `maxArray(marks, 5)` | 50 |
| Double every element | `doubleAll(marks, 5)` | `marks` becomes 20 40 60 80 100 |

- Write `arr[]` in the parameter list; pass just the name `marks` in the call, with no brackets.
- Unlike a single `int`, changes to array elements inside the function **do** stay after it returns.
- Pass the real size, or the loop may read past the end of the array.

## 7. Common mistakes and practice

**Common mistakes**

- Missing prototype when `main()` appears before the function definition.
- Forgetting `return` in a non-`void` function.
- Putting a `;` after the function header in the definition: `int sum(int a, int b); { ... }`.
- Expecting a parameter to change the caller's variable; it is only a copy.

**Practice problems**

1. Write `isPrime(int n)` that returns 1 if `n` is prime, else 0.
2. Write `power(int base, int exp)` using a loop.
3. Write `maxArray(int arr[], int n)` that returns the largest element.
4. Write `average(int arr[], int n)` that returns the average of an array.

## 8. Quick check: predict the output

Cover the answers, work out what each program prints, then check.

**Puzzle A**

```
void change(int x) { x = 99; }

int main(void) {
    int n = 5;
    change(n);
    printf("%d", n);
}
```

**Puzzle B**

```
void tick(void) {
    static int t = 0;
    t++;
    printf("%d ", t);
}

int main(void) {
    tick(); tick(); tick();
}
```

| Puzzle | Output | Why |
|---|---|---|
| A | `5` | `x` is a copy; the original is untouched |
| B | `1 2 3` | `static` keeps `t` alive between calls |

**Remember:** a function is a vending machine. Arguments in, work hidden, value out. Arguments are copies, so your own data stays safe; arrays are the exception, because the function works on the original.

## Next to learn

- Recursion
- Pointers